

Modelos de Deep Learning

Transformers

Universidad ORT Uruguay

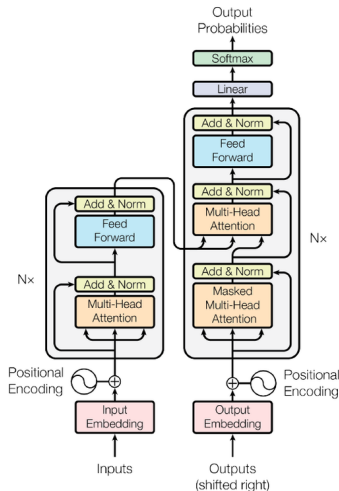
17 de Noviembre, 2025

Arquitectura original del Transformer de [Vaswani et al., 2017]

- Ver también [Alammar, 2018]
- **Encoder** (izquierda) y **Decoder** (derecha).
- El encoder aplica N veces el mismo **bloque**:

Multi-Head Attention $\rightarrow$ Add & Norm $\rightarrow$
Feed-Forward $\rightarrow$ Add & Norm.

- El decoder aplica N veces un bloque similar:
 - **Masked** Multi-Head Attention sobre los outputs previos,
 - Multi-Head Attention sobre las salidas del encoder.
- Al final **lineal** + **softmax** sobre el vocabulario.



Embeddings y Positional Encoding

Sea una secuencia de entrada $\mathbf{x} = (x_1, \dots, x_{T_x})$, $x_t \in \mathcal{V}$ (vocabulario).

- Cada token se mapea a un embedding: $\mathbf{e}_t = E(x_t) \sim (d_{\text{model}})$.
- Se le suma un **positional encoding** $\text{PE}_t \sim (d_{\text{model}})$:

$$\mathbf{z}_t^{(0)} = \mathbf{e}_t + \text{PE}_t.$$

En forma matricial:

$$\mathbf{Z}^{(0)} = \begin{bmatrix} \left(\mathbf{z}_1^{(0)}\right)^\top \\ \vdots \\ \left(\mathbf{z}_{T_x}^{(0)}\right)^\top \end{bmatrix} \sim (T_x, d_{\text{model}}).$$

Esta matriz es la entrada a la primera capa del encoder.

El bloque de encoder

Para la capa $l = 1, \dots, N$ del encoder, la entrada es $\mathbf{Z}^{(l-1)} \sim (\mathcal{T}_x, d_{\text{model}})$.

1. Multi-Head Self-Attention:

$$\mathbf{H}^{(l)} = \text{MHA}(\mathbf{Z}^{(l-1)}, \mathbf{Z}^{(l-1)}, \mathbf{Z}^{(l-1)}).$$

2. Residual + LayerNorm:

$$\tilde{\mathbf{Z}}^{(l)} = \text{LayerNorm}(\mathbf{Z}^{(l-1)} + \mathbf{H}^{(l)}).$$

3. Feed-Forward:

$$\mathbf{F}^{(l)} = \text{FFN}(\tilde{\mathbf{Z}}^{(l)}).$$

4. Residual + LayerNorm:

$$\mathbf{Z}^{(l)} = \text{LayerNorm}(\tilde{\mathbf{Z}}^{(l)} + \mathbf{F}^{(l)}).$$

Multi-Head Attention

Dadas matrices

$$\mathbf{Q} \sim (m, d_{\text{model}}), \quad \mathbf{K} \sim (n, d_{\text{model}}), \quad \mathbf{V} \sim (n, d_{\text{model}}),$$

cada **head** $i = 1, \dots, h$ usa proyecciones lineales:

$$\mathbf{Q}_i = \mathbf{Q}\mathbf{W}_i^Q, \quad \mathbf{K}_i = \mathbf{K}\mathbf{W}_i^K, \quad \mathbf{V}_i = \mathbf{V}\mathbf{W}_i^V,$$

con

$$\mathbf{W}_i^Q, \mathbf{W}_i^K \sim (d_{\text{model}}, d_k), \quad \mathbf{W}_i^V \sim (d_{\text{model}}, d_v).$$

Multi-Head Attention

- Cada head calcula Scaled Dot-Product Attention:

$$\text{head}_i = \text{Softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_k}}\right) \mathbf{V}_i \sim (m, d_v).$$

- Luego se concatenan los resultados de las h heads y se proyectan:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O,$$

donde $\mathbf{W}^O \sim (hd_v, d_{\text{model}})$.

- En el **encoder**: $\mathbf{Q} = \mathbf{K} = \mathbf{V}$ (self-attention).
- En el **decoder**: hay self-attention y atención sobre las salidas del encoder.

Self-Attention en el encoder

- En el encoder, se usa **self-attention**:

$$\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{Z}^{(l-1)} \sim (T_x, d_{\text{model}}).$$

- Entonces cada head produce:

$$\text{head}_i^{(l)} = \text{Softmax}\left(\frac{\mathbf{Z}^{(l-1)}\mathbf{W}_i^Q(\mathbf{Z}^{(l-1)}\mathbf{W}_i^K)^\top}{\sqrt{d_k}}\right)\mathbf{Z}^{(l-1)}\mathbf{W}_i^V.$$

- Cada fila de $\text{head}_i^{(l)}$ es una combinación lineal de las filas de $\mathbf{Z}^{(l-1)}$.
- La concatenación de todas las heads y la proyección final dan $\mathbf{H}^{(l)}$.

Definición de Layer Normalization

- Sea un vector de activación $\mathbf{u} = (u_1, \dots, u_d)$.
- **LayerNorm** normaliza el vector **a lo largo de sus coordenadas** (no en batch).
- Se calculan:

$$\mu = \frac{1}{d} \sum_{i=1}^d u_i \quad (\text{media})$$

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (u_i - \mu)^2 \quad (\text{varianza})$$

- Luego se normaliza cada coordenada: $\hat{u}_i = \frac{u_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$.

Definición de Layer Normalization

- Luego se aplica un **re-escalado y desplazamiento** con parámetros entrenables:

$$\text{LayerNorm}(\mathbf{u}) = \gamma \odot \hat{\mathbf{u}} + \beta,$$

donde $\gamma, \beta \sim (d)$.

- Para una secuencia representada como una matriz $\mathbf{U} \sim (T, d)$,

$$\text{LayerNorm}(\mathbf{U}) = \begin{bmatrix} \text{LayerNorm}(\mathbf{u}_1) \\ \vdots \\ \text{LayerNorm}(\mathbf{u}_T) \end{bmatrix}.$$

Feed-Forward Network (FFN)

- Se usa una FNN aplicada **independientemente** a cada posición de la secuencia (misma red para todos los tokens).
- En la versión original se usa un MLP de 2 capas.
- Para un vector $\mathbf{z} \sim (d_{\text{model}})$ la FFN se define como:

$$\text{FFN}(\mathbf{z}) = \sigma(\mathbf{z}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2,$$

- $\mathbf{W}_1 \sim (d_{\text{model}}, d_{\text{ff}})$, $\mathbf{W}_2 \sim (d_{\text{ff}}, d_{\text{model}})$,
- σ es una activación no lineal (ReLU original, GELU hoy día.).

Feed-Forward Network (FFN)

- En forma matricial, si la entrada a la capa es $\mathbf{Z} \sim (T, d_{\text{model}})$, donde cada fila corresponde a una posición de la secuencia, entonces:

$$\text{FFN}(\mathbf{Z}) = \begin{bmatrix} \text{FFN}(\mathbf{Z}_{1,:}) \\ \vdots \\ \text{FFN}(\mathbf{Z}_{T,:}) \end{bmatrix}.$$

- Es decir:
 - La misma MLP se aplica **fila a fila**.
 - No hay interacción entre posiciones en la FFN: toda la mezcla entre tokens viene de la atención.

Tokens especiales en el decoder: <BOS> y <EOS>

- **Entrada al decoder durante entrenamiento:**

$$(\text{<BOS>}, y_1, y_2, \dots, y_{T_y-1})$$

- **Salida esperada del decoder:**

$$(y_1, y_2, \dots, y_{T_y}, \text{<EOS>})$$

- **En inferencia**, el decoder comienza con <BOS> y genera tokens secuencialmente hasta producir <EOS> .

Decoder: entradas y máscara causal

- La entrada del decoder es la secuencia de salida desplazada:

$$\mathbf{y} = (y_1, \dots, y_{T_y}), \text{ se usa } (<\text{BOS}>, y_1, \dots, y_{T_y-1}).$$

- Se obtienen embeddings + positional encodings:

$$\mathbf{z}_{\text{dec}}^{(0)} \sim (T_y, d_{\text{model}}).$$

- En la primera subcapa se aplica **Masked Multi-Head Self-Attention**:

$$\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{z}_{\text{dec}}^{(l-1)}.$$

- La máscara causal hace que, para la posición t , la softmax ignore entradas correspondientes a posiciones futuras $> t$:

$$(\mathbf{QK}^\top)_{tj} \rightarrow \begin{cases} -\infty & j > t, \\ (\mathbf{QK}^\top)_{tj} & j \leq t. \end{cases}$$

Decoder: atención sobre las salidas del encoder

- Queries: salidas de la subcapa anterior del decoder: $\mathbf{Q} = \tilde{\mathbf{Z}}_{\text{dec}}^{(l)} \sim (T_y, d_{\text{model}})$.
- Keys/Values: salidas finales del encoder: $\mathbf{K} = \mathbf{V} = \mathbf{Z}^{(N)} \sim (T_x, d_{\text{model}})$.
- Se aplica Multi-Head Attention:

$$\mathbf{H}_{\text{enc-dec}}^{(l)} = \text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}),$$

- Seguido de:

$$\hat{\mathbf{Z}}_{\text{dec}}^{(l)} = \text{LayerNorm}(\tilde{\mathbf{Z}}_{\text{dec}}^{(l)} + \mathbf{H}_{\text{enc-dec}}^{(l)}),$$

- Y luego una FFN + Add & Norm análoga al encoder.

Salida final: proyección y softmax

■ La salida de la última capa del decoder es $\mathbf{z}_{\text{dec}}^{(N)} \sim (T_y, d_{\text{model}})$.

■ Para cada posición t :

$$\mathbf{o}_t = \mathbf{z}_{\text{dec},t}^{(N)} \mathbf{W}_{\text{out}} + \mathbf{b}_{\text{out}} \sim (|\mathcal{V}|)$$

donde $\mathbf{W}_{\text{out}} \sim (d_{\text{model}}, |\mathcal{V}|)$.

■ Luego:

$$p(y_t \mid y_{<t}, \mathbf{x}) = \text{Softmax}(\mathbf{o}_t)$$

Observaciones sobre la secuencia de entrada

- Sea una secuencia de entrada de longitud T_x : $\mathbf{X} \sim (T_x, d_{\text{model}})$
- El encoder puede procesar cualquier longitud T_x porque el self-attention se define para cualquier dimensión con $\mathbf{Q}, \mathbf{K}, \mathbf{V} \sim (T_x, d_{\text{model}})$.
- No hay recurrencia en el tiempo.
- La información posicional se incorpora sumando:

$$\mathbf{z}_t = \mathbf{e}_t + \text{PE}_t, \quad 1 \leq t \leq T_x,$$

válida para cualquier longitud.

- Para batches con distintas longitudes se usa **padding** que luego se enmascara para que no intervenga en el attention.

Observaciones sobre la secuencia de salida

- El decoder genera la secuencia **autoregresivamente**: $p(y_t \mid y_{<t}, \mathbf{x})$,
- Usa una **máscara causal** que impide acceder al futuro.
- La generación continúa hasta producir el token especial <EOS>:

$$y_1, y_2, \dots, y_{T_y}, \text{<EOS>}.$$

- La atención encoder-decoder permite relacionar: $T_y \neq T_x$, de modo que el largo de la salida es **variable** y determinado por el modelo.

Positional Encoding: Motivación

El mecanismo de atención es **invariante al orden** de los tokens:

la salida depende solo de productos escalares entre filas de $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, sin noción temporal.

Se suma un **vector de posición** a cada embedding:

$$\mathbf{z}_t = \mathbf{e}_t + \text{PE}_t, \quad 1 \leq t \leq T.$$

El vector $\text{PE}_t \sim (d_{\text{model}})$ debe tener la misma dimensión que los embeddings.

Positional Encoding Sinusoidal

El **positional encoding sinusoidal** define para cada posición $\text{pos} = t$ y cada coordenada $i \in \{0, \dots, d_{\text{model}} - 1\}$:

$$\text{PE}_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right),$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right).$$

- Las frecuencias decrecen geométricamente con la dimensión.
- No requiere parámetros.

Bibliografía

-  Alammar, J. (2018).
The illustrated transformer.
[Blog post.](#)
-  Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017).
Attention is all you need.
In Advances in Neural Information Processing Systems, volume 30. Curran Associates, Inc.